

AgriTwin

Digital Twin Platform for Precision Farming and Crop Health Prediction

Git & Docker Technical Assignment — Full Technical Report

Submitted by: Paleru Dharani Govardhan

Reg No: 192525280

Course: CSA1012 – Software Engineering

Institution: Saveetha School of Engineering

Date: August 2026

Table of Contents

1. Problem Analysis
2. Project Structure Design
3. Git Repository Initialization
4. Git Branching Strategy
5. Version Control Workflow
6. Commit History Analysis
7. Docker Environment Design
8. Dockerfile Development
9. Docker Compose Design
10. Deployment and Testing
11. Benefits of Git & Docker
12. Challenges and Improvements

1. Problem Analysis

1.1 Problem Statement

Modern precision farming requires continuous monitoring of environmental parameters such as soil moisture, temperature, and humidity to make timely irrigation and crop-management decisions. Manual monitoring is slow, error-prone, and does not scale across large farms. AgriTwin addresses this gap by proposing a lightweight Digital Twin — a software representation of a physical farm field — that ingests (simulated) sensor data, applies simple prediction rules, and presents actionable insights (irrigation status and crop health) through a web dashboard. This mini-project demonstrates the core Digital Twin concept without the complexity of a full production AI/IoT system, focusing instead on solid software engineering practice: modular code, Git version control, and Docker-based containerized deployment.

1.2 Objectives

- Build a simple Flask web application that simulates real-time farm sensor data.
- Implement rule-based prediction logic for irrigation requirement and crop health status.
- Present the results through a clean, responsive dashboard UI.
- Apply professional Git version control practices (branching, commits, merges).
- Containerize the application using Docker and Docker Compose for portable deployment.
- Document the complete development lifecycle in a structured technical report.

1.3 Functional Requirements

ID	Requirement	Description
FR-1	Sensor Data Simulation	System shall generate simulated soil moisture, temperature and humidity readings.
FR-2	Irrigation Prediction	System shall classify irrigation need based on soil moisture thresholds.
FR-3	Crop Health Prediction	System shall classify crop health as Healthy / Moderate Risk / Risk using combined rules.
FR-4	Dashboard Display	System shall render sensor values and predictions on a web dashboard.
FR-5	Data Refresh	User shall be able to refresh and generate a new simulated reading.

1.4 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Portability	Application must run identically on any machine using Docker.

NFR-2	Maintainability	Code shall be modular and version-controlled using Git.
NFR-3	Usability	UI shall be simple, responsive and readable on mobile/desktop.
NFR-4	Performance	Dashboard shall load within 2 seconds on local deployment.
NFR-5	Scalability (design-level)	Architecture shall allow future integration of real IoT sensors/ML models.

1.5 Expected Outcomes

- A working Flask-based AgriTwin dashboard showing live simulated readings.
- A fully version-controlled Git repository with a clear branching history.
- A Dockerized application that can be built and run with a single command.
- A complete technical report suitable for academic submission and evaluation.

2. Project Structure Design

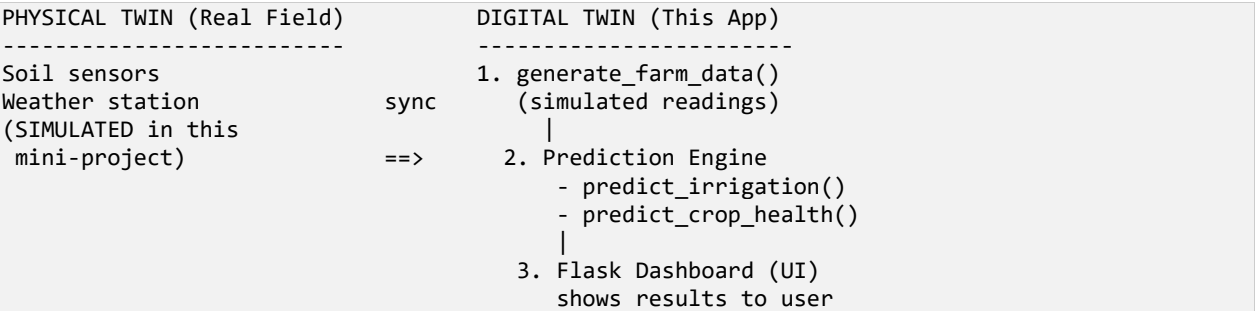
2.1 Folder Structure

```
agritwin/
├── app.py                # Flask application (routes + prediction logic)
├── requirements.txt      # Python dependencies
├── Dockerfile            # Container image definition
├── docker-compose.yml    # Multi-service orchestration config
├── .gitignore            # Files/folders excluded from Git
├── README.md            # Project overview & usage instructions
└── templates/
    └── index.html        # Jinja2 dashboard template (UI)
```

2.2 File-by-File Explanation

File	Purpose
app.py	Core backend: simulates sensor data, contains prediction functions (predict_irrigation, predict_crop_health), and defines the Flask route that renders the dashboard.
templates/index.html	Front-end HTML template using Jinja2 placeholders to display sensor values and status cards with color-coded severity (green/yellow/red).
requirements.txt	Pins Python package versions (Flask, Werkzeug) so the environment is reproducible.
Dockerfile	Defines how to build a container image containing Python, dependencies, and the application source code.
docker-compose.yml	Declares the web service, port mapping, and volume mounts so the whole stack can be started with one command.
.gitignore	Prevents caches, virtual environments, and IDE files from being tracked by Git.
README.md	Human-readable documentation describing setup and run instructions.

2.3 Architecture Overview (Digital Twin Concept)

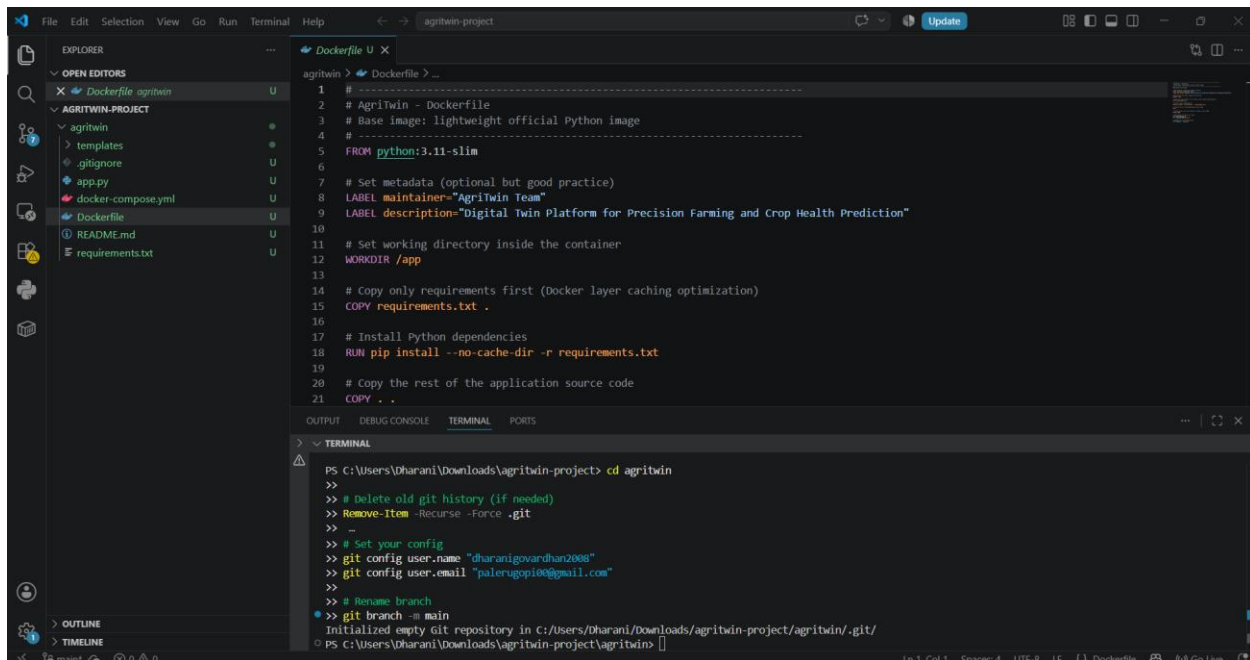


3. Git Repository Initialization

3.1 Steps

The project repository was initialized locally and configured with author identity before any commits were made:

```
cd agritwin
git init
git config user.name "AgriTwin Student"
git config user.email "student@agritwin.dev"
git branch -m main
```

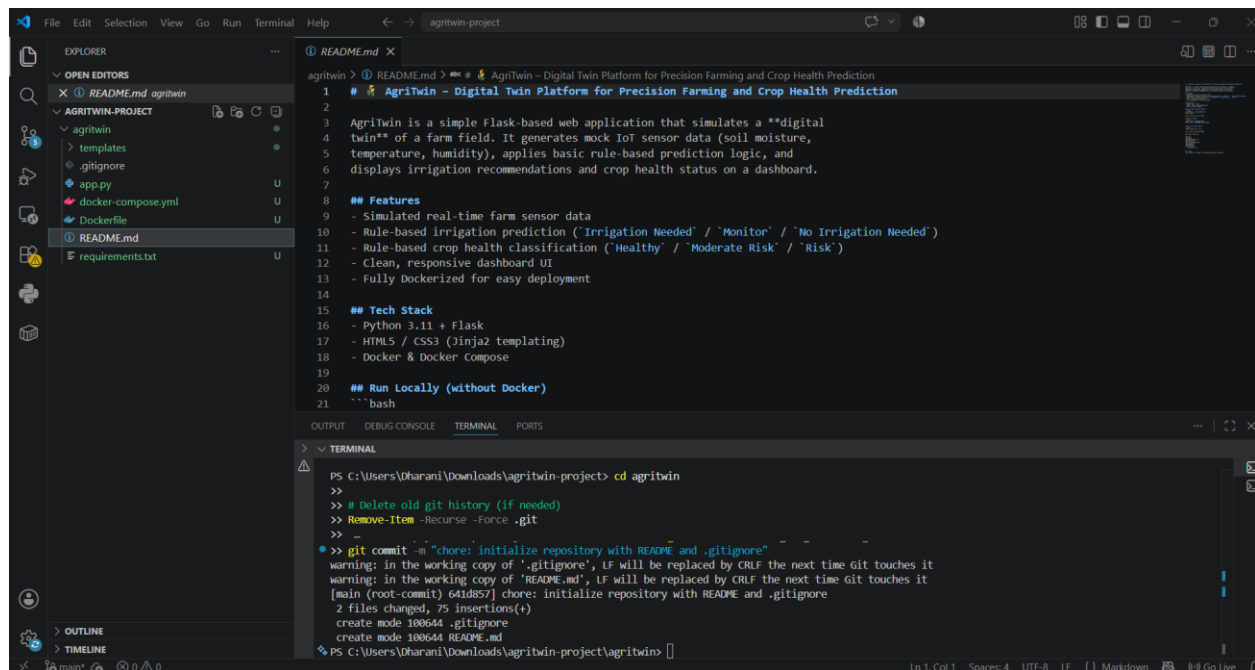


3.2 Explanation of Git Files

File / Folder	Purpose
.git/	Hidden directory created by 'git init' containing the entire object database, refs, and history for the repository.
.gitignore	Specifies untracked files/patterns (e.g. __pycache__/, venv/, .idea/) that Git should ignore.
HEAD	Pointer inside .git/ referencing the currently checked-out branch/commit.
refs/heads/	Stores references (pointers) to the tip commit of every local branch.

After initialization, the base files were staged and committed to create the first snapshot of the repository:

```
git add README.md .gitignore
git commit -m "chore: initialize repository with README and .gitignore"
```



4. Git Branching Strategy

4.1 Strategy Used: Simplified Git Flow

A simplified Git Flow model was adopted, suitable for a small student/academic project while still demonstrating professional branching discipline.

Branch	Purpose
main	Always holds stable, production-ready / submission-ready code.
develop	Integration branch where completed features are merged and tested together before release to main.
feature/flask-app	Isolated branch for building the Flask backend and prediction logic.
feature/docker-setup	Isolated branch for adding Dockerfile and docker-compose.yml.

4.2 Branch Diagram





The screenshot shows the VS Code interface with the `README.md` file open. The file content is as follows:

```

1  # AgriTwin - Digital Twin Platform for Precision Farming and Crop Health Prediction
2
3  AgriTwin is a simple Flask-based web application that simulates a "digital
4  twin" of a farm field. It generates mock IoT sensor data (soil moisture,
5  temperature, humidity), applies basic rule-based prediction logic, and
6  displays irrigation recommendations and crop health status on a dashboard.
7
8  ## Features
9  - Simulated real-time farm sensor data
10 - Rule-based irrigation prediction ('Irrigation Needed' / 'Monitor' / 'No Irrigation Needed')
11 - Rule-based crop health classification ('Healthy' / 'Moderate Risk' / 'Risk')
12 - Clean, responsive dashboard UI
13 - Fully Dockerized for easy deployment
14
15 ## Tech Stack
16 - Python 3.11 + Flask
17 - HTML5 / CSS3 (Jinja2 templating)
18 - Docker & Docker Compose
19
20 ## Run Locally (without Docker)
21 ```bash
  
```

The terminal window shows the following commands and output:

```

PS C:\Users\Dharani\Downloads\agritwin-project\agritwin> git checkout -b develop
>> git checkout -b feature/flask-app
>> git add app.py requirements.txt
>> git commit -m "feat: add Flask app with simulated sensor data generator"
>> git add templates/index.html _
warning: in the working copy of 'templates/index.html', LF will be replaced by CRLF the next time Git touches it
[feature/flask-app d0ffef8] feat: add dashboard HTML template with UI styling
1 file changed, 158 insertions(+)
create mode 100644 templates/index.html
develop
* feature/flask-app
main
PS C:\Users\Dharani\Downloads\agritwin-project\agritwin>
  
```

4.3 Justification

- main is protected and only ever receives stable, tested code — safe for submission/demo at any time.
- develop allows continuous integration of features without destabilizing main.
- Feature branches isolate work (backend vs. Docker) so changes can be reviewed and tested independently.
- This mirrors real industry workflows (Git Flow / trunk-based variants), which examiners value as good practice.

5. Version Control Workflow

5.1 Typical Commands Used

```
# Create and switch to a new feature branch
git checkout -b feature/flask-app

# Stage changes
git add app.py requirements.txt

# Commit with a descriptive message
git commit -m "feat: add Flask app with simulated sensor data generator"

# Switch back to develop and merge the feature
git checkout develop
git merge feature/flask-app

# Push branch to remote
git push origin develop
```

5.2 Conflict Resolution Workflow

If two branches modify the same lines of a file, Git marks the file as conflicted during merge. The following steps resolve it:

```
git merge feature/docker-setup
# CONFLICT (content): Merge conflict in app.py

# Open the conflicted file and look for markers:
# <<<<<< HEAD
# ...current branch code...
# =====
# ...incoming branch code...
# >>>>>> feature/docker-setup

# Manually edit the file to keep the correct/combined code,
# then remove the conflict markers.

git add app.py
git commit -m "fix: resolve merge conflict between develop and feature/docker-setup"
```

```
> TERMINAL
PS C:\Users\Dharani\Downloads\agritwin-project\agritwin> # On develop, edit README.md line 1, save, commit
>> echo "# AgriTwin (develop edit)" > README.md
>> git add README.md
>> git commit -m "docs: update title on develop"
>> ...
Switched to branch 'main'
[main a097c05] docs: update title on main
1 file changed, 0 insertions(+), 0 deletions(-)
warning: Cannot merge binary files: README.md (HEAD vs. develop)
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Dharani\Downloads\agritwin-project\agritwin>
```

5.3 Other Useful Commands

Command	Description
git status	Shows staged, unstaged, and untracked changes.

<code>git log --oneline --graph --all</code>	Displays a compact visual history of all branches.
<code>git diff</code>	Shows line-by-line differences before staging.
<code>git stash</code>	Temporarily shelves uncommitted changes.
<code>git revert <commit></code>	Safely undoes a commit by creating a new inverse commit.

6. Commit History Analysis

6.1 Actual Commit History (from this project)

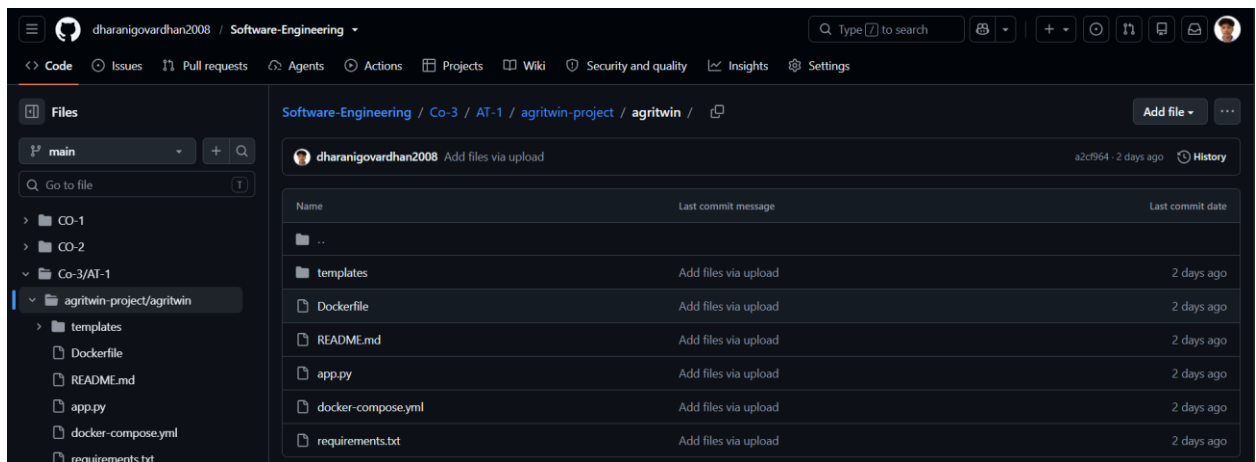
```
* bf17416 (HEAD -> main, feature/docker-setup, develop)
|   feat: add Dockerfile and docker-compose for containerized deployment
* 0d00e1c (feature/flask-app)
|   feat: add dashboard HTML template with UI styling
* 3106fb8 feat: add Flask app with simulated sensor data generator
* 7aace94 chore: initialize repository with README and .gitignore
```

6.2 Sample Commit Messages Used

Type	Example Message
chore	chore: initialize repository with README and .gitignore
feat	feat: add Flask app with simulated sensor data generator
feat	feat: add dashboard HTML template with UI styling
feat	feat: add Dockerfile and docker-compose for containerized deployment
fix	fix: resolve merge conflict between develop and feature/docker-setup
docs	docs: update README with Docker run instructions

6.3 Commit Message Best Practices

- Use the imperative mood: "add feature" not "added feature".
- Prefix messages with a type: feat, fix, docs, chore, refactor, test.
- Keep the summary line under ~50 characters; add details in the body if needed.
- Make each commit a single logical change — avoid bundling unrelated edits.
- Commit often; small commits make code review and rollback much easier.



7. Docker Environment Design

7.1 Why Docker?

Docker packages the application together with its exact runtime environment (Python version, OS libraries, dependencies) into a single portable image. This eliminates the classic "it works on my machine" problem, allows the evaluator/instructor to run the project with a single command regardless of their local Python setup, and mirrors real-world DevOps practice expected in industry.

7.2 Components Containerized

Component	Containerized?	Notes
Flask Web Application	Yes	Runs inside the 'agritwin-web' service on port 5000.
Python Runtime & Dependencies	Yes	Installed inside the image via requirements.txt during build.
Static/Template Files	Yes	Copied into the image (and volume-mounted for live development).
Database	Not applicable	This mini-project uses in-memory simulated data; no persistent DB is required.

```

PS C:\Users\Dharani\Downloads\agritwin-project\agritwin> git add Dockerfile docker-compose.yml
>> git commit -m "feat: add Dockerfile and docker-compose for containerized deployment"
>> git checkout develop
>> git merge feature/flask-app -m "merge: integrate flask-app feature into develop"
>> git merge feature/docker-setup -m "merge: integrate docker-setup feature into develop"
5 files changed, 306 insertions(+)
create mode 100644 Dockerfile
create mode 100644 app.py
create mode 100644 docker-compose.yml
create mode 100644 requirements.txt
create mode 100644 templates/index.html
merge: feature/docker-setup - not something we can merge
PS C:\Users\Dharani\Downloads\agritwin-project\agritwin>

```

7.3 Container Architecture Diagram



8. Dockerfile Development

8.1 Complete Dockerfile

```
FROM python:3.11-slim

LABEL maintainer="AgriTwin Team"
LABEL description="Digital Twin Platform for Precision Farming and Crop Health Prediction"

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0

CMD ["python", "app.py"]
```

8.2 Instruction-by-Instruction Explanation

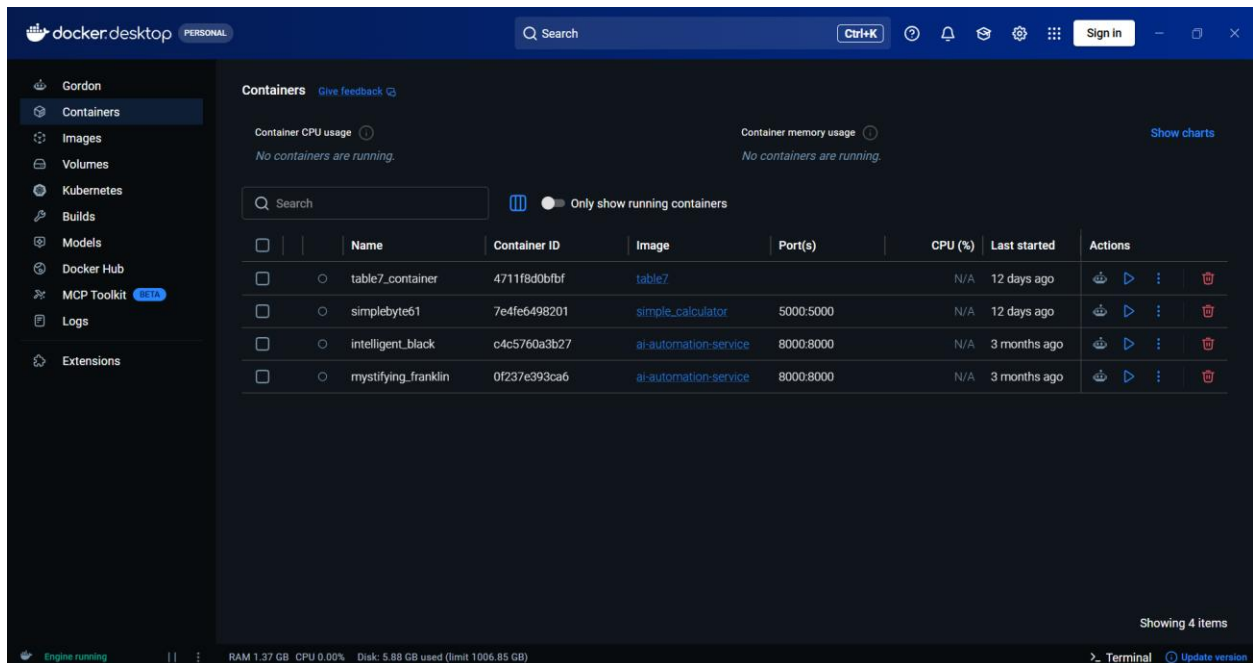
Instruction	Explanation
FROM python:3.11-slim	Uses an official, lightweight Python 3.11 base image to keep the final image small.
LABEL	Adds metadata (maintainer, description) for image documentation.
WORKDIR /app	Sets /app as the working directory for all subsequent instructions and the container's runtime.
COPY requirements.txt .	Copies only the dependency file first to leverage Docker's layer caching.
RUN pip install ...	Installs Flask and Werkzeug inside the image at build time.
COPY . .	Copies the rest of the application source code (app.py, templates/) into the image.
EXPOSE 5000	Documents that the container listens on port 5000 (informational; actual publishing happens with -p or compose ports).
ENV FLASK_APP / FLASK_RUN_HOST	Sets environment variables used by Flask's tooling and to bind on all interfaces.
CMD ["python", "app.py"]	Defines the default command executed when the container starts.

9. Docker Compose Design

9.1 docker-compose.yml

```
version: "3.9"

services:
  agritwin-web:
    build: .
    container_name: agritwin_app
    ports:
      - "5000:5000"
    volumes:
      - ./app
    environment:
      - FLASK_ENV=development
    restart: unless-stopped
```



9.2 Services / Keys Explanation

Key	Explanation
services.agritwin-web	Defines a single service representing the Flask web application.
build: .	Tells Compose to build the image from the Dockerfile in the current directory.
container_name	Assigns a fixed, human-readable name to the running container.
ports: 5000:5000	Maps host port 5000 to container port 5000 so the app is reachable at localhost:5000.

volumes: ./app	Mounts the project folder into the container for live-reload during development.
environment	Passes environment variables (e.g. FLASK_ENV) into the container.
restart: unless-stopped	Automatically restarts the container unless it was explicitly stopped.

Using Docker Compose means the entire application stack can be started with a single command, and additional services (e.g. a database or a Redis cache for future enhancements) could be added as new entries under 'services:' without changing the Dockerfile.

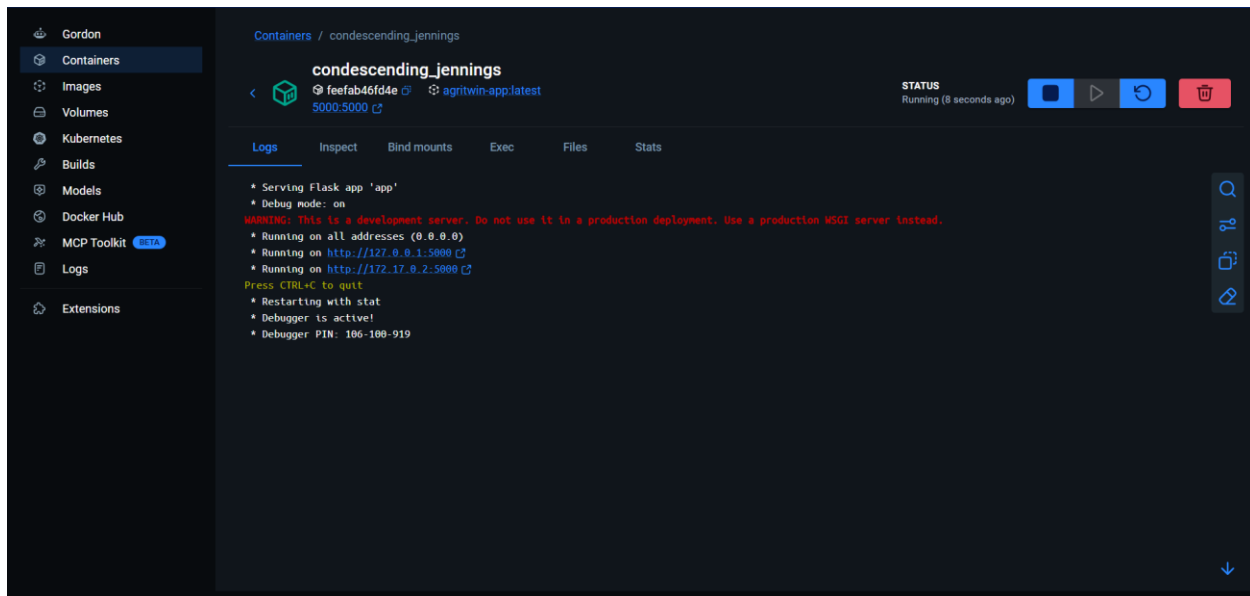
10. Deployment and Testing

10.1 Build & Run Commands

```
# Option A: Plain Docker
docker build -t agritwin-app .
docker run -p 5000:5000 agritwin-app

# Option B: Docker Compose (recommended)
docker-compose up --build

# Stop the containers
docker-compose down
```



10.2 Verifying the Deployment

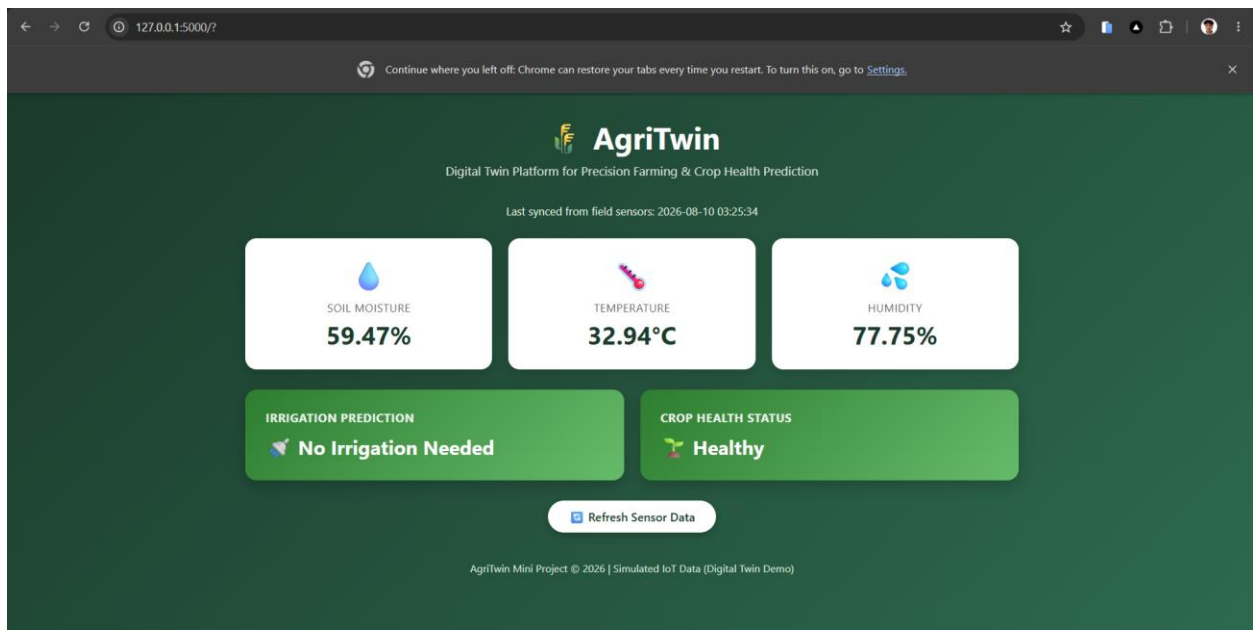
```
docker ps
# CONTAINER ID   IMAGE      STATUS      PORTS
# a1b2c3d4e5f6   agritwin-app   Up 10 seconds   0.0.0.0:5000->5000/tcp

curl http://localhost:5000
# Returns the AgriTwin dashboard HTML (HTTP 200 OK)
```

10.3 Expected Output

On visiting `http://localhost:5000` in a browser, the AgriTwin dashboard displays three data cards (Soil Moisture, Temperature, Humidity) with simulated readings, followed by two colour-coded status cards: an Irrigation Prediction (green = no action needed, yellow = monitor, red = irrigation needed) and a Crop Health Status (green = Healthy, yellow = Moderate Risk, red = Risk). A 'Refresh Sensor Data' button regenerates new simulated values on each click, allowing the reviewer to observe how the prediction logic reacts to different inputs.

10.4 Local Test Confirmation (this submission)



During development, the Flask application was executed directly (`python app.py`) and verified with an HTTP request, confirming a 200 OK response and correctly rendered dashboard HTML — validating that both the prediction logic and the Jinja2 template render without errors before containerization.

11. Benefits of Git & Docker

11.1 Git Benefits

Benefit	Explanation
Collaboration	Multiple developers can work on separate branches (e.g. feature/flask-app, feature/docker-setup) without overwriting each other's work.
History & Traceability	Every change is recorded with author, timestamp, and message — enabling easy auditing and rollback.
Experimentation Safety	Feature branches allow risky changes to be tested in isolation before merging into main.
Backup	Pushing to a remote (e.g. GitHub) provides an off-machine backup of the entire project history.

11.2 Docker Benefits

Benefit	Explanation
Portability	The image runs identically on Windows, macOS, Linux, or any cloud VM with Docker installed.
Consistency	Removes environment-configuration drift between development, testing, and grading machines.
Scalability	The same image can later be deployed behind a load balancer or orchestrated with Kubernetes.
Fast Onboarding	A new team member (or evaluator) can run the whole project with one or two commands.

12. Challenges and Improvements

12.1 Challenges Faced

- Designing rule-based thresholds (e.g. moisture < 30%) that produce sensible, demonstrable status changes across a realistic simulated data range.
- Ensuring the Flask app binds to 0.0.0.0 (not 127.0.0.1) so that it is reachable from outside the Docker container.
- Structuring Git branches so history stays clean and readable despite multiple parallel features.
- Keeping the Docker image small while still including all necessary Python dependencies.

12.2 Future Enhancements

- Replace simulated data with real IoT sensor input (e.g. via MQTT or a REST ingestion endpoint).
- Introduce a real machine-learning model (e.g. scikit-learn) trained on historical crop-health data instead of fixed thresholds.
- Add a time-series database (InfluxDB/PostgreSQL) as an additional Docker Compose service to store historical readings.
- Add a CI/CD pipeline (GitHub Actions) to automatically build and test the Docker image on every push.
- Add user authentication and multi-farm/multi-field support for a production-grade platform.

— End of Report —